Assignment 04 Filesystems

1 File systems

1. **Name two factors that are important in the design of a file system.**
   1 Reliability: A user's data should be safely stored even though the machine is turned off, its operating system crashes or the file system itself crashes.
   2 Performance: For programs to use data, they must be able to read, write, update and delete (CRUD) the data, and for large amount of data, this process must be fast.

Both of these factors is important to make the user satisfied.
2. **Name some examples of file metadata.**
A file's metadata is information about the file that is managed by the operating system, but not including the file content (file data). It is like the HTTP header data, but for files.

Example of a file's metadata: size, modification time, owner, security information

2 Files and directories

1. **Consider a Fast File System (FFS) like Linux's ext4.**
   (a) Explain the difference between a hard link and a soft link in this file system. What is the length of the content of a soft link file?
   The difference between the type of links in the FFS is whether they map multiple entities or singular entities and mapping to names vs mapping to numbers

Hard link maps between a file/directory name and the file number, allowing multiple path names for the same file. On the other hand soft link map one file/directory name to another name, rather than a number.

The length of the content of a soft link file is number of characters the path of the original file it points to, which is the file's size.
https://www.geeksforgeeks.org/linux-unix/soft-hard-links-unixlinux/

(b) What is the minimum number of references (hard links) for any given folder?
1 because when a file is created, it gets a reference count to one. When the reference count is zero, the file is removed.

(c) Consider a folder /tmp/myfolder containing 5 subfolders. How many references (hard links) does it have? Try it yourself on a Linux system and include the output. Use ls -ld/tmp/myfolder to view the reference count (hint, it's the second column in the output).
Input:

```
ls -ld tmp/myfolder
```

Output:

```
drwxr-x--x 7 trile eall_s 4096 Oct 25 22:45 tmp/myfolder
```

Total hardlinks in the folder is 7, because we have 5 subdirectories + 2 (tmp and myfolder).
(d) Explain how spatial locality is acheived in a FFS.
FFS uses a number of *locality heuristics* to achieve spatial locality. Locality heuristics are file system block allocation technique that places files nearby disk sectors if they are likely to be read/written at the same time.

2. **NTFS - Flexible tree with extents**
   (a) Explain the differences and use of resident versus non-resident attributes in NTFS.
   Resident attributes store its content directly in the MFT record while a non-resident stores extent pointer in the MFT record and stores the content in those extent pointers.

How it is used: Large attributes that are too big to fit in the MFT are stored as a non-resident, but attributes that fits are stored as a resident attribute. This approach is often used in huge files. For small file, the data attribute structure can be used to store the file content in the MFT record as a resident attribute.

(b) Discuss the benefits of NTFS-style extents in relation to blocks used by FAT or FFS.
The benefit of extends is the variable-size implementation which can hold variable number of blocks. Thus it support huge files, even multi-gigabyte file can be represented by the extent pointers.

While block is defined as a fixed-size chunk of data storage, from 256KB to 4MB, which can only hold data that is in that interval.

The NTFS-style extents is therefore more scalable and robust for increase in file size.
3. **Explain how copy-on-write (COW) helps guard against data corruption.**
When updating an existing file, COW does not overwrite the existing data/metadata, but instead write new versions to new locations, which means changes are written on a newly allocated block. This implementation works like version control, providing a snapshot of any given file. If

data corruption happens, such as crashing while writing to the file, the old file still lives and can be retrieved instead of overwriting both old and new memory.

3 Security

1. **Authentication**

   (a) Why is it important to hash passwords with a unique salt, even if the salt can be publicly known?

   Salt is a random string that is added to the password before hashing. Its function is to make the hash unique. If a salt is not added, different users can have their password have the same hash. The salt is meant to ensure that the hash is unique compared with another user who has the same hashed password.

A *public* salt is not the most optimal choice at first glance, but the purpose of the salt to make uniquely hashed password, so if an attacker does crack a password, the user's salted password cannot be reused against another user. If the salt is not implemented, the attacker can just iterate and compare all the password with the cracked one and have a chance of cracking another. But why not keep salt private/secret? A secret salt is called pepper, but it is kind of a hassle to set up and too may additional secrets may be referred to as "overengineering" it. https://security.stackexchange.com/questions/3272/password-hashing-add-salt-pepper-or-is-salt-enough

(b) Explain how a user can use a program to update the password database, while at the same time does not have read or write permissions to the password database file itself. What are the caveats of this?

A user can update its password DB through a program called SUID (Set user identity document) bit set. When a user executes this program, usually with `passwd [options] [username]` the program runs with the owner perms (typically root) instead of user perms. This means that even if the user lacks write and read access to the DB, the SUID program temporarily runs with elevated privileges and can modify the DB.

The caveats, which i interpret as vulnerability/weaknesses of this program is the use of elevated perms. This makes the program more vulnerable for exploitation because bugs can lead to attacker gaining unathorized access to the root.

https://www.geeksforgeeks.org/linux-unix/passwd-command-in-linux-with-examples/

2. **Software vulnerabilities**

   (a) Describe the problem with the well-known gets() library call. Name another library call that is safe to use that accomplishes the same thing.

   `gets()` is a function used to read a line of input from standard input (stdin) into a character

array. But it has been deprecated and removed because of its unsafe behaviour. This is because `gets()` does not limit the number of characters read, which can lead to buffer overflows if the input exceeds the allocated array size. It also does not handle errors or EOF (end-of-file) efficiently. Because it is deprecated and removed, the usage of `gets()` is also not stable.

`fgets()` is the safer alternative
https://www.geeksforgeeks.org/c/gets-in-c/

(b) Explain why a microkernel is statistically more secure than a monolithic kernel.

A monolithic kernel runs the entire OS on a single program (in a single address space) in kernel mode. On the other hand, the microkernel implements user and kernel services in a single address space with only the most essential functions and components are in the kernel.

Since all the components can interact directly the monolithic kernel is faster than the microkernel. But the microkernel abstracts all the important functions and have a system that is less complex than the monolithic kernel, the microkernel is more secure, less vulnerable for bugs and errors, and easier to debug. The microkernel is also easier to scale, thus it makes it easier to add security measures.

https://www.geeksforgeeks.org/operating-systems/difference-between-microkernel-and-monolithic-kernel/